Integración de notificaciones por WhatsApp

Implementación observada y split de desarrollo para otro sistema
Noviembre de 2026

Resumen ejecutivo

La solución usa una cuenta de WhatsApp vinculada por código QR a un servicio Node.js con Baileys. La aplicación registra cada aviso en PostgreSQL y solicita al puente que procese la cola. El puente conserva la sesión, envía mensajes y actualiza su estado. Baileys se conecta como dispositivo vinculado; esta arquitectura no corresponde a WhatsApp Business Platform.

Mapa de integración

Evento de negocio pago o boletín	**Aplicación** valida y encola	**PostgreSQL** cola y sesión	**Puente Baileys** procesa y registra	**WhatsApp** destinatario

Flujo: el sistema crea una notificación con clave de evento única; invoca POST /dispatch; el puente también revisa la cola cada 30 segundos. Cuando la sesión está conectada, toma hasta 20 avisos pendientes por ciclo y registra el resultado.

Qué mensajes se generan hoy

Disparador	Condición	Resultado
Comprobante de pago	Compra o servicio con número válido	Aviso de recepción y revisión pendiente
Aprobación de pago	Administrador aprueba inscripción pendiente	Aviso de aprobación y acceso
Boletín	Administrador envía campaña; cliente con consentimiento y compra aprobada	Novedad, enlace a cursos e instrucción para darse de baja

Los enlaces wa.me que aparecen en otras partes del sitio abren conversaciones iniciadas por el visitante. Son distintos de este sistema de envíos automáticos.

Componentes y contrato mínimo

Componente	Responsabilidad / interfaz
Aplicación de negocio	Recoge número internacional y consentimiento separado; crea avisos transaccionales o de marketing dentro del flujo de negocio.
Base de datos	Guarda contacto y consentimiento, estado de autenticación de Baileys, vinculación y cola de notificaciones. eventKey es único para impedir encolados duplicados.
Puente Node.js	Ofrece /health y rutas protegidas por clave para estado, QR, desconexión y /dispatch; envía por Baileys.
Panel administrativo	Solo administradores consultan conexión, generan QR y desconectan la cuenta. La aplicación actúa como proxy y no expone la clave al navegador.

Modelo de cola

WhatsAppNotification conserva: identificador, eventKey, userId opcional, teléfono, texto, tipo (TRANSACTIONAL o MARKETING), estado, intentos, disponibilidad, bloqueo, identificador del mensaje, error y fechas. Los estados observados son pending, sending, sent, cancelled y needs_review.

WhatsAppAuthState almacena las credenciales y claves de la sesión en PostgreSQL. WhatsAppConnection registra la vinculación. En otro sistema, estos nombres pueden cambiar, pero deben mantenerse esas responsabilidades.

Reglas de envío y fallos

• El número se valida en formato internacional con prefijo + y se almacena normalizado; para enviar se utilizan solo dígitos.

• El puente vuelve a comprobar el consentimiento y la compra aprobada justo antes de cada mensaje de marketing. Si ya no corresponden, cancela ese aviso.

• Cada aviso pendiente se reclama con bloqueo de fila; al enviar se guarda sentAt y messageId. Los envíos se espacian 1,5 segundos.

• Un error durante el envío, o un reinicio con un aviso en sending, deja el registro en needs_review. No se reenvía automáticamente porque el resultado podría ser ambiguo.

• Si el puente está desconectado o suspendido, los avisos permanecen en pending y se procesan al reconectar. La aplicación puede seguir registrando el evento.

Despliegue observado

El repositorio define el puente como servicio web de Render con Node.js; usa una conexión PostgreSQL directa. La aplicación configura WA_BRIDGE_URL y WA_BRIDGE_API_KEY; el puente usa DATABASE_URL y BRIDGE_API_KEY. Las claves compartidas deben coincidir. Tras migrar la base, el administrador vincula la cuenta escaneando el QR desde Dispositivos vinculados.

Split de desarrollo para otro sistema

La secuencia propuesta reproduce el comportamiento existente. “Responsable” indica el perfil que lidera la tarea; una misma persona puede cubrir varios perfiles.

Fase / responsable	Entregable	Depende de
1. Modelo y datos · Backend / BD	Campos de teléfono y consentimiento; tablas de sesión, conexión y cola; índices y migración reversible.	Acceso a la base del sistema destino
2. Eventos de negocio · Backend	Puntos de encolado para comprobante, aprobación y campaña; plantillas de texto; claves de evento únicas.	Fase 1 y eventos definidos
3. Servicio puente · Backend / integración	Servicio Node.js con Baileys, sesión persistente, QR, API con clave, despacho y estados.	Fase 1 y entorno de ejecución
4. Panel y consentimiento · Frontend	Captura y edición del número y permiso; vista administrativa de estado, QR y desconexión.	Fases 2 y 3
5. Infraestructura · DevOps	Despliegue del puente, conexión directa a BD, variables secretas, health check y vinculación de la cuenta.	Fase 3
6. Validación · QA + equipo	Pruebas de negocio, consentimiento, cola, reconexión, errores y revisión manual; verificación en entorno previo a producción.	Fases 1 a 5

Adaptación al sistema destino

• Identificar los eventos equivalentes a pago presentado, pago aprobado y campaña; mapearlos a usuarios y números de teléfono existentes.

• Mantener la creación del evento de negocio y del aviso en una misma transacción cuando compartan base de datos; si están en sistemas distintos, definir un mecanismo fiable de publicación.

• Si el sistema no tiene panel administrativo, crear una interfaz restringida para vincular la cuenta y consultar el estado. Nunca entregar la clave del puente al cliente web.

• Mantener la separación entre avisos de operación y marketing; ofrecer baja de marketing y verificarla al momento de enviar.

Criterios de aceptación

Escenario	Resultado esperado
Alta de pago	Una única notificación pending por evento; número normalizado y aviso enviado cuando el puente esté conectado.
Aprobación duplicada	No se duplica el evento ni el mensaje de aprobación.
Campaña sin consentimiento o sin compra aprobada	No se envía el boletín; una baja posterior al encolado cancela el aviso pendiente.
Puente fuera de línea	La cola conserva los avisos; al reconectar se reanuda el procesamiento.
Fallo o reinicio durante el envío	El aviso queda en needs_review con error visible para revisión humana; no se reenvía automáticamente.
Acceso al QR y desconexión	Solo el administrador puede operar la vinculación; solicitudes sin clave al puente reciben rechazo.

Operación y límites

Se debe observar la salud del servicio, el estado de vinculación, el volumen de pending y needs_review y los errores de envío. La revisión de needs_review exige comprobar si el destinatario recibió el mensaje antes de decidir un reenvío manual. La implementación actual depende de Baileys y de una sesión de dispositivo vinculado; conviene evaluar su mantenimiento y las políticas aplicables antes de usarla a gran escala.